

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «ОМО»

Тема: «Сравнение классических методов классификации»

Выполнил:

Студент 3-го курса

Группы АС-66

Ануфриенко М. А.

Проверил:

Крощенко А. А.

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 1

- Iris
- Определить вид ириса (setosa, versicolor, virginica) по измерениям его цветка

Задания:

1. Загрузите данные и ознакомьтесь с ними;
2. Разделите выборку на обучающую и тестовую;
3. Обучите модели k-NN, Decision Tree и SVM. Для k-NN попробуйте найти оптимальное значение k;
4. Оцените точность (accuracy) каждой модели на тестовой выборке;
5. Сравните результаты и сделайте вывод, какая модель лучше всего справилась с этой задачей.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Загрузка датасета
df = pd.read_csv("iris.csv")

target_col = "variety"

X = df.drop(columns=[target_col])
y = df[target_col]

# 2. Разделение данных на train/test выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# -----
# 3. Обучение трех моделей
# -----

# --- k-NN с k=5 (базовое значение)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

```

# --- Decision Tree
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)

# --- SVM
svm = SVC(kernel="rbf", random_state=42)
svm.fit(X_train, y_train)

# -----
# 4. Подбор оптимального k
# -----

k_values = range(1, 31)
accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    accuracies.append(accuracy_score(y_test, preds))

best_k = k_values[accuracies.index(max(accuracies))]

print("Лучший k:", best_k)

# --- График зависимости точности от k ---
plt.plot(k_values, accuracies)
plt.xlabel("k")
plt.ylabel("Accuracy")
plt.title("Зависимость точности k-NN от k")
plt.grid(True)
plt.show()

```



```
# -----  
# 5. Оценка точности моделей  
# -----  
  
knn_pred = knn.predict(X_test)  
tree_pred = tree.predict(X_test)  
svm_pred = svm.predict(X_test)  
  
acc_knn = accuracy_score(y_test, knn_pred)  
acc_tree = accuracy_score(y_test, tree_pred)  
acc_svm = accuracy_score(y_test, svm_pred)  
  
print("Точность k-NN (k=5):", acc_knn)  
print("Точность Decision Tree:", acc_tree)  
print("Точность SVM:", acc_svm)  
print("Лучшая точность k-NN (подбор k):", max(accuracies))
```

```
Лучший k: 3  
Точность k-NN (k=5): 1.0  
Точность Decision Tree: 0.9333333333333333  
Точность SVM: 0.9666666666666667  
Лучшая точность k-NN (подбор k): 1.0
```

Вывод: я применил алгоритм k-ближайших соседей (k-NN) к датасету Iris, разделили данные на обучающую и тестовую выборки, обучили модель и проверили её качество. Путём подбора значения k мы оценили влияние параметра на точность и получили итоговую ассигуру тестовой выборки. Метод k-ближайших соседей (k-NN) лучше всего подходит для данной выборки.